Министерство науки и высшего образования Республики Казахстан
Алматинский Технологический Университет
Кафедра информационных систем

САМОСТОЯТЕЛЬНАЯ РАБОТА СТУДЕНТА

по дисциплине «Проектирование и применение NoSQL-технологий»

Вариант 11. Кэш расписания университета (Redis + MongoDB)

Выполнил: студент 3 курса ОП «Информационные системы»
Евгений Л, группа ИС 24-21

Преподаватель: Рашидов Дамир

2026

1. Цель и постановка задачи

Цель работы - спроектировать и реализовать прототип NoSQL-подсистемы кэширования расписания университета: MongoDB как основное хранилище, Redis как кэш готовых расписаний, с TTL и удалением кэша после изменения расписания, и провести анализ устаревших данных (stale data).

Постановка: в университете есть группы, преподаватели, аудитории и занятия. Студенты много раз в день открывают расписание своей группы, преподаватели - свое, деканат смотрит занятость аудиторий и иногда переносит пары. Расписание группы собирается из четырех коллекций, и делать эту сборку на каждый запрос расточительно. Нужно, чтобы повторные чтения шли из кэша, а после переноса пары студент при следующем запросе уже видел новую аудиторию.

2. Требования

2.1 Функциональные

1. Показать расписание группы на неделю и на конкретный день.
2. Показать расписание преподавателя.
3. Показать занятость аудитории.
4. Добавить, перенести (аудитория, день, время) и удалить занятие.
5. После изменения занятия следующий запрос возвращает новое расписание, а не старое из кэша.
6. Найти свободные аудитории в заданный день и время.
7. Посчитать недельную нагрузку преподавателей и найти конфликты аудиторий.
8. Показать статистику кэша: попадания, промахи, число инвалидаций.

2.2 Нефункциональные

1. Повторное чтение расписания из кэша быстрее 1 мс (из MongoDB сборка занимает 3-20 мс).
2. Окно устаревших данных после переноса пары - ноль при штатной инвалидации и не больше TTL (300 с) в худшем случае.
3. Кэш не хранит ничего эксклюзивно: при недоступности Redis или истечении ключа данные собираются из MongoDB.
4. Воспроизводимость: запуск с нуля через docker compose и скрипт заполнения, секретов в репозитории нет (.env.example).

3. Предметная область и Access Patterns

Пользователи: студент (читает расписание группы), преподаватель (читает свое), деканат (смотрит аудитории, переносит пары, ищет свободные аудитории и конфликты), кафедра (нагрузка преподавателей).

#	Запрос	Кто	Частота	Приоритет	Где выполняется	Ожидаемый результат
AP1	расписание группы на неделю	студент	очень часто	высокий	Redis, при промахе Mongo	занятия группы, отсортированы по дню и времени
AP2	расписание группы на день	студент	очень часто	высокий	фильтр по недельному кэшу	занятия одного дня
AP3	расписание преподавателя	преподаватель	часто	высокий	Redis / Mongo	все пары по всем группам
AP4	занятость аудитории	деканат	средне	средний	Redis / Mongo	все пары в аудитории
AP5	перенести пару	деканат	редко	высокий	Mongo + DEL ключей	расписания группы, преподавателя, старой и новой аудитории обновлены
AP6	добавить / удалить пару	деканат	редко	средний	Mongo + DEL ключей	то же
AP7	свободные аудитории в слот	деканат	редко	средний	Mongo, $lookup	аудитории без занятий в день и время
AP8	нагрузка преподавателей	кафедра	раз в семестр	низкий	Mongo, $group	пары и часы в неделю
AP9	конфликты аудиторий	деканат	после правок	средний	Mongo, $group	две группы в одной аудитории одновременно
AP10	поиск по предмету	студент	редко	низкий	Mongo, regex	занятия по предмету

AP1-AP4 - чтение, именно они кэшируются. AP5-AP6 - запись, они сбрасывают кэш. AP7-AP10 - аналитика, выполняется редко и в кэше не нуждается.

4. Сравнение альтернатив и ADR

Вариант	Плюсы	Минусы
A. Только MongoDB с	одна система, данные всегда свежие	каждый запрос собирает расписание через $lookup из 4 коллекций, в пик (утро, перемена)

индексами		это лишняя работа на каждое открытие страницы
B. Только Redis	очень быстро	нет сложных запросов (свободные аудитории, конфликты, нагрузка), данные плоские, основное хранилище из Redis делать неудобно
C. PostgreSQL + кэш	привычные JOIN	расписанию жесткая схема не нужна, а кэш все равно придется добавить, то есть тот же вариант D, только с SQL
D. MongoDB + Redis (cache-aside)	Mongo хранит и отвечает на сложные запросы, Redis отдает готовое расписание за доли миллисекунды	две системы, нужно следить за согласованностью кэша

Решение (ADR-001, docs/ADR.md): вариант D. MongoDB - основное хранилище, Redis - кэш готовых расписаний по шаблону cache-aside: читаем Redis, при промахе собираем из Mongo и кладем с TTL 300 с; при любом изменении занятия удаляем ключи группы, преподавателя и аудитории, причем и старые, и новые значения. Расписание на день отдельно не кэшируется, оно фильтруется из недельного в приложении, чтобы ключей было меньше и инвалидация была проще.

Отвергнут write-through (сразу писать в кэш новое расписание): при переносе одной пары пришлось бы пересобирать до шести расписаний, а изменений мало и их читают не сразу. Проще удалить ключ и собрать по первому запросу.

Последствия: горячие чтения уходят в Redis, Mongo нагружена только промахами и записью. Минус - окно устаревших данных не нулевое при гонке читателя и писателя, поэтому TTL оставлен небольшим как страховка (см. раздел 8).

5. Схема данных и архитектура

MongoDB, база `university`, четыре коллекции:

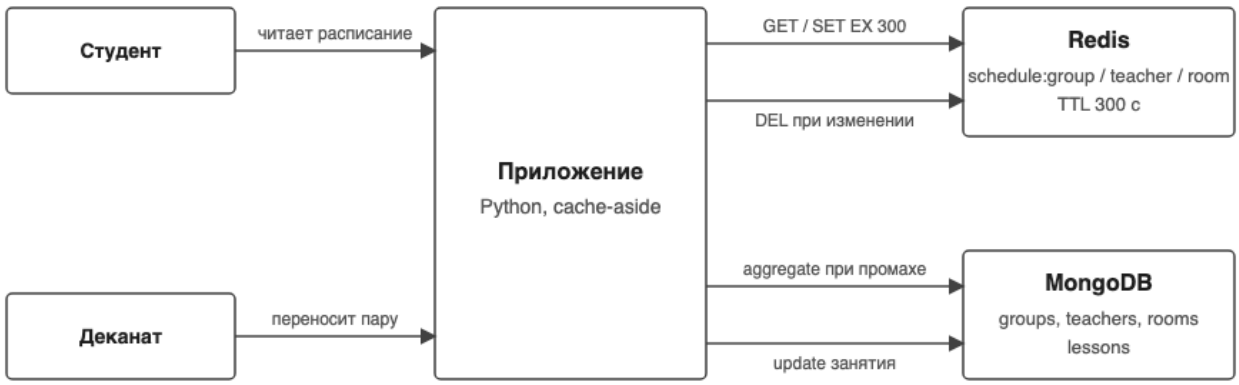
```
groups   { code: "IS-31", course: 3, faculty: "Информационные системы", students:
24 }
teachers { teacher_id: 2, name: "Нурланова Д.К.", department: "Информационные
системы" }
rooms    { room: "B-201", building: "B", capacity: 25, type: "lab" }
lessons  { lesson_id: "L001", group_code: "IS-31", subject: "NoSQL технологии",
teacher_id: 2,
           room: "B-202", weekday: 4, start: "14:30", end: "15:50", kind: "лаб" }
```

Занятие ссылается на группу, преподавателя и аудиторию по ключам (referencing): преподаватель переименовывается в одном месте, а готовое расписание с именами собирается через $lookup только при промахе кэша. Индексы: уникальные по lesson_id, code, teacher_id, room и составные (group_code, weekday, start), (teacher_id, weekday), (room, weekday, start) - по одному под каждый кэшируемый запрос.

Redis:

Ключ	Значение	TTL
schedule:group:IS-31	JSON расписания группы на неделю (с именем преподавателя и корпусом)	300 с
schedule:teacher:2	JSON расписания преподавателя	300 с
schedule:room:B-201	JSON занятости аудитории	300 с
stats:hits, stats:misses, stats:invalidations	счетчики INCR	нет

Данные: 6 групп, 8 преподавателей, 7 аудиторий, 40 занятий (data/*.json, все синтетические). Одно пересечение по аудитории (B-202, четверг 14:30) заложено специально, чтобы запрос конфликтов что-то находил.



Чтение: GET ключа в Redis, при промахе aggregate в Mongo и SET ключа с EX 300. Запись: update в Mongo, затем DEL затронутых ключей.

6. Реализация

Python 3.9, pymongo, redis-py, pytest. Структура: `src/schedule_cache/` - config (переменные из .env), db (подключения), repository (Mongo), cache (Redis), service (cache-aside и инвалидация); `scripts/` - seed, CLI, демо-запросы, эксперимент; `tests/` - 14 тестов.

Cache-aside в сервисе:

```
def _cached(self, key, loader):
    t0 = time.perf_counter()
    data = self.cache.get(key)          # GET + stats:hits / stats:misses
    source = "cache"
    if data is None:
        data = loader()                 # aggregate в Mongo
        self.cache.set(key, data)       # SET key json EX ttl
```

```
        source = "mongo"
    return data, source, round((time.perf_counter() - t0) * 1000, 2)

def get_group_schedule(self, code):
    return self._cached(ScheduleCache.group_key(code), lambda:
self.repo.group_schedule(code))
```

Инвалидация: ключи считаются и от старого, и от нового документа занятия.

```
@staticmethod
def keys_for(lesson):
    return [ScheduleCache.group_key(lesson["group_code"]),
            ScheduleCache.teacher_key(lesson["teacher_id"]),
            ScheduleCache.room_key(lesson["room"])]

def change_lesson(self, lesson_id, changes):
    old = self.repo.update_lesson(lesson_id, changes)   # find_one_and_update,
возвращает старый документ
    new = self.repo.get_lesson(lesson_id)
    return self._drop(old, new)                         # DEL всех ключей из old
и new
```

Сборка расписания в Mongo (то, что кладется в кэш):

```
[{"$match": {"group_code": code}},
 {"$lookup": {"from": "teachers", "localField": "teacher_id", "foreignField":
"teacher_id", "as": "t"}}, {"$unwind": "$t"},
 {"$lookup": {"from": "rooms", "localField": "room", "foreignField": "room",
"as": "r"}}, {"$unwind": "$r"},
 {"$project": {"_id": 0, "lesson_id": 1, "group_code": 1, "weekday": 1, "start":
1, "end": 1,
               "subject": 1, "kind": 1, "room": 1, "building": "$r.building",
"teacher": "$t.name"}},
 {"$sort": {"weekday": 1, "start": 1}}]
```

Запуск: `cp .env.example .env; docker compose up -d; pip install -r requirements.txt; python scripts/seed.py`. CLI: `python scripts/schedule.py group IS-31 [--day 2] | teacher 2 | room B-201 | move L009 --room B-202 | add ... | remove L041 | stats | keys`.

7. Запросы и результаты

Вывод `python scripts/demo_queries.py` после seed.

Q1. Расписание группы IS-31, два раза подряд. Первый запрос - промах, сборка в Mongo, второй - попадание.

```
источник: mongo, 8.44 ms, 8 занятий
  Пн 08:00-09:20  NoSQL технологии      лаб       B-201  Нурланова Д.К.  IS-31
  Пн 13:00-14:20  Английский язык       практика  B-305  Омарова А.Н.    IS-31
  Вт 08:00-09:20  Проектирование ИС     практика  A-205  Сериков А.Б.    IS-31
```

```
  Вт 14:30–15:50  Базы данных           лекция    С–110  Сериков А.Б.    IS–31
  Чт 14:30–15:50  NoSQL технологии      лаб       В–202  Нурланова Д.К.  IS–31
  Пт 08:00–09:20  Физическая культура   практика  GYM    Исаева М.О.     IS–31
  Пт 13:00–14:20  Операционные системы  лаб       В–202  Жумабеков Р.М.  IS–31
  Сб 14:30–15:50  Веб–программирование  лаб       В–201  Ахметов Е.С.    IS–31
второй раз: источник cache, 0.51 ms, ttl ключа 300 с
```

Q2. IS-31 на вторник. День берется из недельного ключа, отдельного ключа на день нет.

```
источник: cache, 0.49 ms, 2 занятий
  Вт 08:00–09:20  Проектирование ИС     практика  А–205  Сериков А.Б.    IS–31
  Вт 14:30–15:50  Базы данных           лекция    С–110  Сериков А.Б.    IS–31
```

Q3. Расписание преподавателя 2 (Нурланова Д.К.).

```
источник: mongo, 3.59 ms, 5 занятий
  Пн 08:00–09:20  NoSQL технологии  лаб  В–201  IS–31
  Вт 14:30–15:50  NoSQL технологии  лаб  В–201  IS–32
  Чт 08:00–09:20  NoSQL технологии  лаб  В–202  IS–33
  Чт 14:30–15:50  NoSQL технологии  лаб  В–202  IS–31
  Сб 09:30–10:50  NoSQL технологии  лаб  В–201  IT–41
```

Q4. Занятость аудитории B-201 - 7 пар за неделю, источник mongo, 3.05 ms (далее этот ключ используется в Q8).

Q5. Свободные аудитории в четверг 14:30 (нетривиальный: $lookup с pipeline и $expr по rooms, оставляем те, у кого массив busy пустой).

```
  А–205  корпус А,  30 мест, practice
  В–201  корпус В,  25 мест, lab
  В–305  корпус В,  30 мест, practice
  С–110  корпус С,  60 мест, lecture
  GYM    корпус С,  40 мест, sport
```

Заняты A-101 и B-202.

Q6. Нагрузка преподавателей (нетривиальный: $group по teacher_id с $addToSet групп, $lookup имени, часы = пары * 80 / 60).

```
  Сериков А.Б.      7 пар   9.3 ч/нед  5 групп
  Жумабеков Р.М.    6 пар   8.0 ч/нед  6 групп
  Омарова А.Н.      6 пар   8.0 ч/нед  6 групп
  Ахметов Е.С.      5 пар   6.7 ч/нед  5 групп
  Бекетов Т.Ж.      5 пар   6.7 ч/нед  3 групп
  Нурланова Д.К.    5 пар   6.7 ч/нед  4 групп
  Исаева М.О.       4 пар   5.3 ч/нед  4 групп
  Касымова Г.Т.     2 пар   2.7 ч/нед  2 групп
```

Q7. Конфликты аудиторий ($group по room+weekday+start, count > 1).

```
  В–202 Чт 14:30: группы ['IS–31', 'IT–41'], занятия ['L001', 'L040']
```

Q8. Перенос L001 из B-202 в B-201 и инвалидация.

```
  было: IS-31 Чт 14:30 B-202
  ключи в кэше до:    ['schedule:group:IS-31', 'schedule:room:B-201',
'schedule:teacher:2']
  сброшены:           ['schedule:group:IS-31', 'schedule:teacher:2',
'schedule:room:B-202', 'schedule:room:B-201']
  ключи в кэше после: []
  стало: IS-31 Чт 14:30 B-201  [источник mongo]
```

Сброшены четыре ключа: группа, преподаватель, старая аудитория B-202 (ее в кэше не было, DEL просто вернул 0) и новая B-201, которая в кэше была и стала бы устаревшей.

Q9. Поиск по предмету (regex "nosql", без учета регистра) - 5 занятий: L008, L001, L009, L016, L037.

Q10. Статистика кэша после демо: hits 2, misses 4, invalidations 8, hit_ratio 0.333 (в демо почти каждый запрос новый, поэтому низкий; на реальной нагрузке см. раздел 8).

8. Анализ stale data

`python scripts/stale_analysis.py`, занятие L002 группы IS-31, аудитория C-110 переносится в A-205, измеряется, сколько секунд кэш продолжает отдавать C-110.

Сценарий	Что делается	Кэш отдавал старую аудиторию
1. Только TTL	TTL=5 с, после изменения ключ не удаляем	4.89 с, до истечения TTL
2. TTL + DEL	TTL=300 с, после изменения DEL ключей (штатный режим)	0.00 с
3. Гонка	читатель прочитал из Mongo до записи, писатель обновил и сделал DEL, читатель положил в кэш старое	4.88 с, спас только TTL=5 с
4. Нагрузка	300 случайных чтений по 6 группам, TTL=300	hit ratio 0.98; mongo 3.69 мс, redis 0.34 мс

Выводы. Без удаления ключа студент видит старую аудиторию весь TTL, а при рабочем TTL в 300 с это пять минут - для переноса пары недопустимо. С удалением ключа окно нулевое: следующий запрос после переноса сразу идет в Mongo. Сценарий 3 показывает, почему TTL нельзя убирать совсем: если запрос на чтение и перенос пары пришли одновременно, читатель может положить в кэш уже устаревшее расписание, и без TTL оно жило бы вечно. Поэтому TTL оставлен коротким, он работает как страховка. На случайной нагрузке 98% чтений уходят в Redis и отвечают в 10 раз быстрее.

Как закрыть гонку полностью: повторный DEL через секунду после записи (delayed double delete) или хранить в Mongo версию расписания и класть ее в ключ. В прототипе это не делал, оставил как ограничение.

9. Тестирование

14 тестов pytest, ходят в отдельную базу university_test и Redis db 1, перед каждым тестом данные заливаются заново.

```
tests/test_cache.py::test_miss_then_hit PASSED
tests/test_cache.py::test_key_has_ttl PASSED
tests/test_cache.py::test_ttl_expires PASSED
tests/test_cache.py::test_cached_equals_mongo PASSED
tests/test_invalidation.py::test_change_drops_old_and_new_keys PASSED
tests/test_invalidation.py::test_read_after_change_is_fresh PASSED
tests/test_invalidation.py::test_stale_without_invalidation PASSED
tests/test_invalidation.py::test_add_and_remove_invalidate PASSED
tests/test_invalidation.py::test_change_unknown_lesson_raises PASSED
tests/test_repository.py::test_seed_counts PASSED
tests/test_repository.py::test_crud_lesson PASSED
tests/test_repository.py::test_group_schedule_sorted_and_joined PASSED
tests/test_repository.py::test_free_rooms_excludes_busy PASSED
tests/test_repository.py::test_conflicts_found PASSED
============================== 14 passed in 4.07s ==============================
```

Сценарии: (1) промах, потом попадание, счетчики растут; (2) ключ с TTL=1 исчезает через секунду и следующий запрос снова идет в Mongo; (3) перенос пары удаляет ключи группы, преподавателя, старой и новой аудитории, чтение после переноса свежее; (4) без инвалидации кэш отдает старую аудиторию - тест специально фиксирует stale data; (5) CRUD занятия, свободные аудитории, конфликты.

10. Ограничения

- Инвалидацию делает приложение. Если изменить lessons напрямую в Mongo (mongosh), кэш узнает об этом только по TTL.
- Гонка чтения и записи (сценарий 3) не закрыта, окно устаревания до TTL.
- Один экземпляр Redis и Mongo без репликации, это прототип.
- Недели числитель/знаменатель и семестры не моделировались, расписание одно на все недели.
- Статистика hits/misses глобальная, не по ключам.

11. Выводы

Спроектирован и реализован прототип кэша расписания: MongoDB хранит группы, преподавателей, аудитории и занятия и отвечает на аналитические запросы, Redis держит готовые расписания по ключам группы, преподавателя и аудитории с TTL 300 с. Реализован cache-aside с инвалидацией после изменения занятия, причем сбрасываются ключи и старых, и новых значений. Эксперимент показал, что только TTL дает окно устаревания в полный TTL, удаление ключа сводит его к нулю, а гонка читателя и

писателя остается и ограничивается TTL. На случайной нагрузке 98% чтений обслуживает Redis, в 10 раз быстрее Mongo. Код, данные, 10 запросов, 14 тестов и инструкция запуска лежат в репозитории проекта.